



AI Mode ▾

All

Videos

Short videos

Images

More ▾



install pip apple silicon



You do not need to install `pip` separately because it comes pre-bundled with modern Python installations on Apple Silicon Macs. 🤖 Reddit +2

Instead, you should install a native ARM64 version of Python (which automatically includes `pip`) using [Homebrew](#) or the official Python installer. 🤖 www.invisiblefriends.net +1

Follow these steps to set up `pip` correctly on your M-series Mac:

## Step 1: Install Python and Pip

Choose **one** of the two primary methods below:

- **Method A: Via Homebrew (Recommended)**

Open your Terminal app and run the Homebrew package manager command to pull down a native build:

```
bash
```

```
brew install python
```

Use code with caution.



- **Method B: Via Official Installer**

Go to the Official Python Downloads Page and download the latest **macOS universal2 installer**. Run the downloaded package to install Python and `pip` simultaneously. 🤖 www.invisiblefriends.net +1

Ask anything



AI Mode ▾

All

Videos

Short videos

Images

More ▾



Use code with caution.



## 2. Verify Python is running natively on Apple Silicon:

**bash**

```
python3 -c "import platform; print(platform.machine())"
```

Use code with caution.



If it outputs `arm64`, your installation is native. If it says `x86_64`, your Terminal is running under Rosetta 2 emulation. 🍷 Reddit +4

## Step 3: Fix "Command Not Found" Errors

If you type `pip3` and receive a `zsh: command not found` error, your shell does not know where `pip` is hidden. Fix this by forcing Python to trigger its built-in bootstrapping module: 🍷 GeeksforGeeks +4

**bash**

```
python3 -m ensurepip --upgrade
```

Use code with caution.



## Best Practice: Always Use Virtual Environments [🔗](#)

To prevent permission conflicts and messy package updates across your

AI Mode ▾ All Videos Short videos Images More ▾



```
# Activate the workspace  
source env/bin/activate
```

```
# Now you can use standard short commands safely  
pip install <package_name>
```

Use code with caution.



If you are planning to work on data science or machine learning, let me know if you intend to install **TensorFlow**, **PyTorch**, or **scikit-learn**. I can give you the specific configuration commands required to leverage your **Apple Silicon GPU core acceleration**. Medium · Diko Ko +1



```
george@Georges-Mac-mini PythonAudioPlaySampleWav %  
pip3 install p5...
```



Your installation is failing because you are using Apple's default system Python framework bundled inside Xcode, which lacks the proper headers to compile `PyOpenGL-accelerate`. System Python environments are locked and highly prone to compiler mismatches on M-series chips.

You can fix this completely by switching to a modern, isolated environment using one of the solutions below.

## Solution 1: Use a Virtual Environment (Recommended &

AI Mode ▾ All Videos Short videos Images More ▾



```
python3 -m venv venv
```

```
# 2. Activate it
```

```
source venv/bin/activate
```

```
# 3. Upgrade your core setup tools inside the environment
```

```
pip install --upgrade pip setuptools wheel
```

```
# 4. Install p5 safely
```

```
pip install p5
```

Use code with caution.



## Solution 2: Bypass `PyOpenGL-accelerate`

The library `p5` relies on `PyOpenGL`. The `PyOpenGL-accelerate` package is just an optional C-extension meant to speed it up. It is notorious for failing to compile on Apple Silicon, but `p5` will work perfectly fine without it.

You can trick the terminal into bypassing the broken package by running:

**bash**

```
# Install the main required OpenGL package first
```

```
pip3 install --user PyOpenGL PyOpenGL_nodebug
```

```
# Force install p5 while telling it to ignore the broken accelerat
```

```
pip3 install --user p5 --no-deps
```

```
# Manually fetch the rest of the working dependencies
```

```
pip3 install --user numpy vispy skia-python pillow glfw hsluv kiwi
```

AI Mode ▾ All Videos Short videos Images More ▾



## bash

```
# 1. Install standard Python
brew install python

# 2. Close and reopen your terminal, then run:
pip3 install p5
```

Use code with caution.



If you see any **GLFW library errors** or **window creation failures** when you try to run your first `p5` script, let me know! I can show you how to **link the macOS native homebrew graphics drivers** so your sketches render correctly.



```
self.distribution.run_command(command)
```

File ...



Even inside your virtual environment, `pip` is still attempting to compile **PyOpenGL-accelerate**, which fails because its underlying Cython code is completely incompatible with Apple Silicon's compiler requirements.

Because `PyOpenGL-accelerate` is an optional, hard-coded dependency in the `p5` installer, it will block your installation every time it tries to build from source

AI Mode ▾ All Videos Short videos Images More ▾

**bash**

```
# 1. Force install the core package while skipping the broken inst  
pip install p5 --no-deps
```

```
# 2. Manually download all of p5's requirements (skipping the brok  
pip install numpy vispy skia-python pillow glfw hsluv kiwisolver f
```

Use code with caution.



## Step 2: Test the Setup

Verify that Python can import the graphic libraries without complaining about the missing acceleration module:

**bash**

```
python -c "import p5; print('Success!')"
```

Use code with caution.



If you encounter a **"visual layout"** or **"GLFW library missing"** error when running an actual sketch, let me know! I can show you how to **link your Mac's native window manager** via Homebrew.



AI Mode ▾ All Videos Short videos Images More ▾



Press **Ctrl + C** on your keyboard to clear the stuck line and return to your normal `(venv) george@Georges-Mac-mini` prompt.

Once you see your clean prompt, copy and paste these lines **one at a time** (pressing Enter after each) to avoid any formatting or quoting issues:

**bash**

```
pip install p5 --no-deps
```

Use code with caution.



**bash**

```
pip install numpy vispy skia-python pillow glfw hsluv kiwisolver f
```

Use code with caution.



**bash**

```
python -c "import p5; print('Success!')"
```

Use code with caution.



If you receive a **"Success!"** output, your installation is complete. If you get any error messages during the test, please paste them here.

AI Mode ▾ All Videos Short videos Images More ▾



**The installation was successtul!** You can safely ignore the red `ERROR` messages at the bottom.

Those dependency errors only appear because `pip` notices you downloaded slightly newer versions of `vispy`, `pillow`, and `PyOpenGL` than what the older `p5` installer originally asked for. However, these newer versions are fully compatible with Apple Silicon, whereas the older ones are not.

## Step 1: Run the Final Test

To make sure `p5` can load everything properly, run this quick check command in your terminal:

```
bash
```

```
python -c "import p5; print('Everything works!')"
```

Use code with caution.



## Step 2: What to expect next

Because `p5` relies on your Mac's graphics systems to open windows and draw shapes, there is a high chance you will encounter a window layout or `GLFW` error the very first time you try to run an actual code script.

If your script crashes or fails to open a window, you can fix it instantly by installing the native Mac window manager backend via Homebrew:

```
bash
```

AI Mode ▾ All Videos Short videos Images More ▾



packaging.python.org

(venv) george@Georges-Mac-mini

PythonAudioPlaySampleWav % pip install p5Requirement ...



The issue is that you ran `pip install p5` again without any flags. This caused `pip` to re-evaluate the full project definition, see the missing components, and try to build the broken `PyOpenGL-accelerate` from source all over again.

Because we bypassed the standard layout by installing dependencies manually in the previous turn, **p5 is actually already fully installed and ready to use in your (venv)**. You do not need to successfully run `pip install p5` alone.

Run this verification check command to prove that the package can load perfectly right now:

**bash**

```
python -c "import p5; print('P5 is successfully installed and work
```

Use code with caution.



## Why you can ignore this error

You have already manually satisfied every library that `p5` needs to draw.

AI Mode ▾ All Videos Short videos Images More ▾



script containing a basic configuration:

python

```
from p5 import *  
  
def setup():  
    size(400, 400)  
  
def draw():  
    background(255)  
    fill(0)  
    circle(200, 200, 50)  
  
if __name__ == '__main__':  
    run()
```

Use code with caution.



Let me know if running this snippet triggers any **GLFW window context** errors or if the test script outputs a **ModuleNotFoundError**. Proactively tell me **which code editor** you are using so I can make sure your terminal configuration syncs up!



—

AI Mode ▾ All Videos Short videos Images More ▾



your virtual environment was created from the built-in Apple system Python ( /Applications/Xcode.app/... ), it uses `LibreSSL 2.8.3`. The library `urllib3` (downloaded automatically as a sub-dependency of `requests`) requires standard `OpenSSL 1.1.1+` and throws that warning every time you run a script.

This warning will likely block `p5` sketches from making network requests or loading online assets. You can completely resolve this by choosing one of the two solutions below:

### Solution 1: Downgrade `urllib3` (Quickest Fix)

The easiest way to make the warning go away inside your current environment is to downgrade `urllib3` to version `1.26.x`, which fully supports macOS LibreSSL:

```
bash
```

```
pip install "urllib3<2.0"
```

Use code with caution.



### Solution 2: Recreate the Environment with Homebrew (Best Practice)

To prevent future compilation or SSL errors, stop basing your virtual environments on Xcode's system Python. Instead, use a standard, decoupled Homebrew installation:

1. Deactivate and delete your current broken environment:

AI Mode ▾ All Videos Short videos Images More ▾



**bash**

```
brew install python@3.11
```

Use code with caution.



3. Create a brand new virtual environment using the Homebrew path:

**bash**

```
python3.11 -m venv venv  
source venv/bin/activate
```

Use code with caution.



4. Reinstall your workspace packages using the safe bypass flags:

**bash**

```
pip install p5 --no-deps  
pip install numpy vispy skia-python pillow glfw hsluv kiwisolver
```

Use code with caution.



Run your test script again using `python -c "import p5"` to verify everything is quiet.

Once your console is clean, let me know **if you are running your code inside VS**

AI Mode ▾

All

Videos

Short videos

Images

More ▾



(venv) george@Georges-Mac-mini

PythonAudioPlaySampleWav % python -c "import p5; ...



**Perfect! Your terminal console is now 100% clean and `p5` is fully functional with no warnings.**

By downgrading to `urllib3 1.26.20`, you successfully patched the LibreSSL incompatibility without needing to scrap and rebuild your environment. The `P5 is successfully installed and working!` message without any preceding warning blocks means you are fully ready to start coding your audio or visual program.

## Next Steps to Run Your First Script

To verify that the window rendering engine works seamlessly with your Mac's graphics hardware, save this basic template as `main.py` in your `PythonAudioPlaySampleWav` directory:

**python**

```
from p5 import *

def setup():
    # Creates a window workspace
    size(400, 400)

def draw():
    # Clears screen to white and draws a black circle
    background(255)
    fill(0)
    circle(200, 200, 100)
```

AI Mode ▾

All

Videos

Short videos

Images

More ▾



```
python main.py
```

Use code with caution.



If you want to make sure your workspace stays synced, let me know:

- **Which code editor** you are using (like VS Code or PyCharm)?
- If you encounter any **GLFW window creation** errors when launching the script?

I can provide the specific configurations to bind your IDE directly to this functional environment!



How do i import the sound library?

Unlike JavaScript's `p5.sound.js` addon, the Python `p5` package does not have a separate sub-library or special import statement for audio. [GitHub](#) +1

Instead, the core audio loading and playback functions are baked directly into your primary `from p5 import *` package definition! You do not need to `pip install` anything else. [strive](#) [learnpython.strivemath.com](#)

## 1. How to use it in code

AI Mode ▾ All Videos Short videos Images More ▾



```
# Global variable to store the sound data
my_sound = None

def setup():
    global my_sound
    size(400, 400)

    # Place your audio file in the exact same folder as this scrip
    # Supports standard formats like .wav and .mp3
    my_sound = load_sound("sample.wav")

def draw():
    background(240)
    fill(50, 150, 250)
    circle(200, 200, 100)

def mouse_pressed():
    # Plays the audio clip whenever you click the window workspace
    if my_sound:
        print("Playing audio...")
        my_sound.play()

if __name__ == '__main__':
    run()
```

Use code with caution.



## 2. Available Audio Methods

Once you load a sound object into Python, you can manipulate it using these

AI Mode ▾ All Videos Short videos Images More ▾



audio files:

- **FileNotFoundError** : Ensure your `sample.wav` or `music.mp3` track sits inside the **exact same folder directory** as your code script file.
- **ModuleNotFoundError / Driver Error** : If python crashes complaining about sound channels or sound mixers, it means your Mac needs the underlying audio wrapper library. You can resolve this instantly in your terminal by running:

```
bash
```

```
pip install sounddevice soundfile
```

Use code with caution.



Let me know if the window pops up and **if you hear the audio track trigger** when you click the canvas! If it crashes or outputs an error trace, paste the text here so we can patch it.



(venv) george@Georges-Mac-mini

PythonAudioPlaySampleWav % ...



⏮ ⏪ ⏴ ⏵ ⏩ ⏭ ⏮ ⏪ ⏴ ⏵ ⏩ ⏭

AI Mode ▾ All Videos Short videos Images More ▾



**playsound** . Stack Overflow

## Step 1: Install **playsound**

While keeping your **(venv)** active in your terminal, download the library:

**bash**

```
pip install playsound==1.2.2
```

Use code with caution.



*(Note: Explicitly using version **1.2.2** avoids a known bugs list inside version 1.3+ on macOS).*

## Step 2: Use it inside your Sketch Code

You can now import **playsound** separately from your graphics canvas loop. Because **playsound** blocks execution while a sound is playing, you should wrap it inside an event action (like clicking the mouse) so it doesn't freeze your visual frame rates: Stack Overflow

**python**

```
from p5 import *  
from playsound import playsound # Import the independent sound mo  
  
def setup():  
    size(400, 400)
```

AI Mode ▾

All

Videos

Short videos

Images

More ▾



```
playsound("sample.wav")

if __name__ == '__main__':
    run()
```

Use code with caution.



## An Alternative for Advanced Projects (Non-Blocking)

If you need sounds to play in the background *without* temporarily pausing your visual animations, you will need a non-blocking audio engine. You can install

**pygame** instead:

**bash**

```
pip install pygame
```

Use code with caution.



Then rewrite your sound trigger logic to utilize the Pygame mixer:

**python**

```
import pygame
pygame.mixer.init()
my_sound = pygame.mixer.Sound("sample.wav")

# Inside mouse_pressed() you can simply call:
```

AI Mode ▾ All Videos Short videos Images More ▾



```
(venv) george@Georges-Mac-mini  
PythonAudioPlaySampleWav % ...
```



The `AttributeError: 'FreeTypeFont' object has no attribute 'getsize'` error occurs because `p5` relies on older Pillow (PIL) font logic. The `getsize()` method was deprecated and then completely removed in Pillow version 10+. Because you have `pillow 11.3.0` installed, `p5` crashes the moment it tries to render text on your screen.

You can fix this incompatibility immediately by downgrading `pillow` to version `9.5.0` inside your virtual environment.

## The Fix

Run this command in your active terminal:

```
bash
```

```
pip install "pillow<10.0.0"
```

Use code with caution.



## Verify the fix

After the installation completes, clear the cache and run your code again:

AI Mode ▾

All

Videos

Short videos

Images

More ▾



The window should now open up without crashing, allowing you to check and play your audio file!

If you encounter any further errors or if the canvas window doesn't appear properly, let me know:

- Is your `sample.wav` file located in the same directory as the script?
- Do you see a **blank screen** or a frozen window when launching?



```
(venv) george@Georges-Mac-mini  
PythonAudioPlaySampleWav % ...
```



The `ModuleNotFoundError: No module named 'AppKit'` error occurs because the `playsound` library relies on Apple's native **AppKit** framework to trigger audio playback on macOS. To interact with macOS native components, Python requires the **PyObjC** bridge packages.

You can fix this dependency error immediately by installing the official macOS Python bindings inside your virtual environment.

## The Fix

Run this command in your active terminal:

```
bash
```

AI Mode ▾ All Videos Short videos Images More ▾



venny the fox

Run your code script again:

**bash**

```
python /Users/george/Documents/GitHub/PythonAudioPlaySampleWav/ven
```

Use code with caution.



Clicking on the `p5` window canvas should now execute smoothly and play your audio file through your Mac's speakers.

If you run into any **audio format errors** (like sample rate or bit-depth issues with your specific WAV file), let me know:

- Is your `sample.wav` file a standard **16-bit** or **24-bit** track?
- Do you hear **stuttering** or does the app freeze when the sound triggers?

